Development Plan
Keystone

Team 13

Ayush Patel
Ahmed Rashrash
Andrew Habib
Waqar Ul-Hassan

Table 1: Revision History

Date	**Developer(s)**	**Change**
Sept 25th 2026	Ayush Patel	Added intro and Sections 1 - 3
Sept 25th 2026	Andrew Habib	Added PoC Demo Plan Section 9
Sept 25th 2026	Ayush Patel	Added Workflow Plan
Sept 26th 2026	All Team Members	Added Team Meeting and Communication Plan and Team Roles
Sept 26th 2026	All Team Members	Added Expected Technology and AI Use and Team Charter
Sept 26th 2026	Waqar Ul-Hassan	Added reflection information
Sept 26th 2026	Andrew Habib	Added Coding Standard Section 12
Sept 26th 2026	Ahmed Rashrash	Added Project Decomposition and Scheduling section
Sept 27th 2026	Ayush Patel	Added reflection
Sept 28th 2026	Ahmed Rashrash	Added reflection information
Sept 28th 2026	Andrew Habib	Added reflection information
Sept 28th 2026	Ahmed Rashrash	Added GitHub url
Sept 28th 2026	All Team Members	Added AI disclosures
Sept 28th 2026	Ayush Patel	Minor Formatting and Proofreading changes
Oct 2nd 2026	Ayush Patel	Added role backups and Definition of Done clarification
Oct 27th 2026	Andrew Habib	Clarify coverage report generation

This document contains details about the development plan for the project. It includes information about the confidentiality and licensing (sections 1 - 3), team communication and meeting plans (sections 4 - 6), workflow and project decomposition (sections 7 - 8), proof of concept demonstration plan (section 9), expected technology (section 10), coding standard (section 11), and appendices for generative AI use disclosure, reflection, and team charter.

1 Confidential Information?

There is no confidential information to protect in this project.

2 IP to Protect

The intellectual property produced by this project consists solely of written content, namely the code and documentation, which is covered under our copyright license, discussed below.

3 Copyright License

The copyright license adopted by the team for this project is the MIT license. This license provides others the right to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the software, provided that the copyright notice and permission notice are included in all copies. The license also disclaims warranty and limits the authors' liability for any damages arising from use of the software. The copyright license for the project can be found in the team repository.

4 Team Meeting Plan

- **Weekly Team Meeting:** Once a week for one hour – Wednesdays, 2:30pm to 3:30pm, in person. Additional virtual meetings will be scheduled as needed when approaching a deliverable deadline.
- **Meeting Chair:** Waqar Ul-Hassan – prepares and sends out the meeting agenda and leads the meetings.
- **Notetaker:** Ahmed Rashrash – records the meeting minutes using the template below.
- **Backup Chair and Notetaker:** To avoid a single point of failure, each role has a designated backup:
  - If Waqar is absent, Ayush Patel acts as meeting chair.
  - If Ahmed is absent, Andrew Habib acts as notetaker.

  - If a backup is also absent, the remaining available member covers the role, since only one person is left to do so. For example, if Waqar and Ayush are both absent, Andrew becomes meeting chair, because Ahmed is already the notetaker.

- **Absence Handoff:** In non-emergency situations, a member who will be absent from a meeting must notify the team in the Teams chat at least 48 hours before the meeting. If the absent member is the chair or notetaker, this notice also tells the backup to take over the role. The backup uses the same agenda and minutes templates, and the same "Team/Supervisor/TA Meeting" GitHub Issue, as the regular chair or notetaker would. The agenda deadline (24 hours before the meeting) still applies to the backup chair. In emergencies, the member should notify the team as soon as reasonably possible.
- **Supervisor Meetings:** We will meet with Dr. Smith at least once a month, and more frequently as needed. These will primarily occur during lecture time, with virtual meetings as a backup when an in-lecture meeting is not possible.
- **Agenda Requirement:** All meetings (team, TA, supervisor) must have an agenda, prepared and shared with the team at least 24 hours in advance by the chair. The agenda template is as follows:
  - Date, time, attendees
  - Purpose of the meeting
  - Review of previous deliverable/meeting: what went well, what needs improvement
  - Team member updates: completed tasks, outstanding tasks, tasks needing assistance
  - Open questions for supervisor / advisor, TA, or stakeholder (as applicable)
  - Action items and task owners for the next meeting
- **Meeting Minutes Template:** Minutes will be recorded for every team, TA, and supervisor meeting, using the following structure:
  - Date, time, attendees, absentees (with reason if provided)
  - Summary of discussion per agenda item
  - Decisions made
  - Action items, owners, and due dates
  - Open questions carried forward

Minutes will be stored in the description of a "Team Meeting" GitHub Issue created for each meeting, using the provided Team Meeting issue template.

5 Team Communication Plan

- **Primary asynchronous channel:** Microsoft Teams chat, for day-to-day communication and quick questions.
- **Task-level communication:** GitHub – issues, pull requests, and code review comments are used for anything related to a specific task, so discussion stays attached to the relevant work item. Members will -mention each other in PRs/issues, assign issues directly, and request reviews through GitHub's review request feature.
- **Meeting minutes:** Recorded and stored on GitHub (see Team Meeting Plan).
- **Supervisor/stakeholder communication:** Email (Dr. Smith prefers email), routed through the team liaison to keep a single point of contact.
- **Response-time expectation:** Each member will respond to direct communication in the team's Teams chat within 24 hours.

6 Team Member Roles

- **Project-Level Roles:**
  - Team Liaison: Ayush Patel – primary point of contact with the supervisor and course staff.
  - Meeting Chair: Waqar Ul-Hassan – prepares and sends out the meeting agenda and leads the meetings. Backup: Ayush Patel.
  - Notetaker: Ahmed Rashrash – records the meeting minutes using the template in Section 4. Backup: Andrew Habib.
  - GitHub Manager: Andrew Habib – responsible for keeping labels, issue templates, branch protection rules, and the project board consistent and up to date.
- **Documentation Quality Assurance:** Rotates per deliverable alphabetically by first name starting with Ahmed for the first deliverable. The member(s) in this role are responsible for checking the completed deliverable against both the repository checklist and the Avenue rubric before submission, and flagging any gaps to the team with enough time to address them before the deadline. This is in addition to, not a replacement for, normal PR review.

- **Technical Roles:**
  - Frontend developer: Responsible for development of a user friendly interface - All members
  - Backend developer: Responsible for developing the backend and database layer - All members
  - Reviewer (code): all members.
  - Reviewer (documentation): all members.
  - Technical QA: Responsible for doing functional testing after each code functionality related PR is merged. This will rotate for each deliverable and will be the same team member who is also the documentation QA.

  We are intentionally keeping technical roles broad and overlapping at this stage rather than assigning rigid specializations, since the team is still early in the project and all members should be able to work across the stack. Specializations may emerge naturally as the project progresses.
- **Flexibility and Role Changes:** Roles are not fixed for the duration of the project. At the start of each milestone, the team will briefly revisit role assignments based on interest, workload balance, and skills members want to develop, and reassign as needed. This ensures no single member is locked into a role that no longer fits the project's needs or their own goals.
- **Overlap by Design:** Technical and review roles are deliberately assigned to "all" members rather than single individuals, so there is no single point of failure if one member is unavailable during a critical period.

7 Workflow Plan

- **Git Usage:** We will primarily be using GitHub for managing the project files. The feature branch strategy will be used with each member creating a dedicated branch for each task that they work on. This branch can be deleted after the task is completed and merged into the main branch. The member should open a pull request after completion of their assigned task. If multiple members are collaborating on the same task, they will work within the same branch. Once completed, a pull request can be opened which will be reviewed by the other team members.

  Link to GitHub Repository

  Overall Workflow Plan

  1. **Issue Creation:** For each deliverable, we will create many different issues. Each issue equates to a specific task that needs to be

completed. An issue could be about multiple related sections of a deliverable or a single modular piece of code functionality.

2. **Branching:** Each team member will create a branch for each task that they work on. If multiple members collaborate on the same task, they will work on the same branch to avoid merge conflicts.

3. **Commit Process:** All changes will be regularly committed to the respective branches. We will follow the Conventional Commits standard (e.g. `feat:`, `fix:`, `docs:`, `chore:`, `refactor:`) so that commit history is easy to scan and purposeful. Each commit should represent a single, working, well-documented change – no commit farming (i.e. no meaningless, placeholder, or "WIP" commits left in the final history). Commit messages should clearly describe *what* changed and, where not obvious, *why*.

4. **Pull Requests (PRs):** Once a team member has completed a task, a pull request on GitHub will be created. Directly pushing to the main branch is not allowed. A pull request must be created and it must pass all CI checks before it can be merged into the main branch. Every PR must include a description. For straightforward or self-explanatory changes, a brief description is sufficient. For changes where the intent, approach, or impact would not be immediately obvious to a reviewer (particularly code changes), the author must provide a more detailed description covering what was changed, why, and any tradeoffs or design decisions made. This makes it much easier for reviewers to understand the changes and provide meaningful feedback.

5. **Team Member Review:** The PR will be reviewed by at least one other team member to ensure that all of the rubric criteria has been met. If the PR contains code changes, the PR will also be reviewed for code quality and functionality. The reviewer(s) provide feedback through comments. The author of the section should respond or implement any feedback provided by the member who reviewed the submission. If a merge conflict arises during review, it is the responsibility of the branch owner to resolve it (or resolved collaboratively if the branch is shared) before requesting re-review.

6. **Merging:** Once the PR has been reviewed and approved, it will successfully be merged into the main branch. We will not use squash merging, to ensure that all commits are visible in the main branch. To maintain a linear history, we will use rebase and merge.

7. **Definition of Done:** An issue/task is considered complete when: its checklist items are fully addressed, its associated PR has passing CI checks and, where required, at least one approving review, and it has been merged into the main branch. Post-merge functional testing

is a separate follow-up activity and is not part of the Definition of Done (see below).

8. **Post-Merge Functional QA:** After each functionality-related PR is merged, the Technical QA member (see Section 6) performs functional testing on the main branch. This testing should be completed before the end of the current deliverable period, and the result is recorded as a comment on the original issue (pass, or a link to any defect found).
   - **If QA passes:** No further action is needed, and the original issue stays closed.
   - **If QA finds a defect:** The Technical QA member creates a new issue labelled *Bug* (or *Feature* if the gap is missing functionality). The new issue must reference the original issue and the PR that introduced the defect, and the original issue gets a comment linking back to it, so the history stays traceable in both directions. The new issue is added to the same milestone and the GitHub Project board. It is assigned, by default, to the author of the original PR, or to another member if the team agrees.

Branch Protection Rules: To enforce the above process, the following protections will be applied to the main branch:

- No direct pushes to main – all changes must go through a pull request.
- Pull requests must pass all required CI checks before merging.
- Code-related pull requests will additionally require review and approval from a designated code owner (at least one of the 4 team members), enforced via a `CODEOWNERS` file. This file will be created once the project's technology stack has been finalized, so that the file can be made using the relevant language/framework directory structure and file extension.
- Non-code files (e.g. documentation, LaTeX files, images) will not have mandatory reviewer approval enforced through branch protection, so that minor changes such as wording fixes, typos, or adding images are not bottlenecked by teammate availability. The team will still follow the PR review process as a best practice for major documentation-related tasks, even where it is not explicitly enforced.

Project Management: We will be using GitHub Issues to keep track of the following items: tasks, meetings, and other important items. Each issue will be part of a milestone that corresponds to a specific deliverable (i.e. Problem Statement and Development Plan, SRS, etc.) All issues and milestones will be part of a GitHub Project (Keystone Project) in a Kanban style so we can visualize and track the progress of our project.

There are template issues readily available from the course instructor, as shown below, and we will be using these throughout our capstone.

Issue Templates Available:

- Lectures: Used to track team details during lectures (i.e. important notes, attendance)
- Peer Review: Individuals outside of the team review one of our items
- Supervisor Meeting: Agenda template for meetings with supervisors/stakeholders
- TA Meeting: Agenda template for meetings with TA
- Team Meeting: Agenda template for weekly meetings with our team
- Blank Issue: Blank template to be used to create a new issue

Issue Labels Classification:

Below are the set of labels in GitHub Issues that we will be using when creating issues to organize them:

- Bug: Used for issues that indicate something is not working as expected
- Feature: Used for issues that represent any new functionality that provides visible value to users.
- TA Meeting: Used for issues that include meeting notes, feedback and actions from TA sessions.
- Team Meeting: Used for issues where team meeting discussions are noted.
- Supervisor / Stakeholder Meeting: Used for issues where meetings with the project supervisors and stakeholders are documented.
- Documentation: Used for issues corresponding to any work related to documentation, including updates to the ReadMe's and latex files.

- **Use of CI/CD:** We will be using GitHub Actions as our CI/CD platform to ensure that all of our documentation and project files are being built, formatted, and reviewed properly before they are merged into the main branch of the project. Workflows, which are defined in YAML files, will trigger automatically on pushes and pull requests. To ensure consistency across all files, merges require both teammate approval (only for code files, using a CODEOWNERS file) and passing CI checks. While we will still use the PR review process for all major documentation-related tasks, we will not officially enforce teammate approval for documentation-related PRs, to ensure that we are not bottlenecked by teammate availability for minor

changes. CI will also run all unit tests to make sure we are not introducing any regressions. Test failures will cause the CI to fail and prevent merging. Coverage reports will be generated during automated testing and imported into SonarQube for analysis. Falling below the 80% statement coverage target will not automatically fail CI or prevent merging but will be flagged for review. Checks will be added to ensure linting, code quality analysis, and unit test coverage (see Expected Technology and Coding Standard). Enforcing CI checks ensures errors are caught early and also ensures we follow industry-standard practices.

8 Project Decomposition and Scheduling

GitHub Projects

GitHub projects will be the primary place where issues are tracked. Each issue is part of a milestone, which is a group of issues that have an explicit due date. Issues move through the stages using a Kanban-style board in the following order: Backlog → Ready → In-Progress → In-Review → Done.

Each issue has a title, a description if needed, a set of assignees, priority level, and classification (feature, bug, task, documentation, meeting). Milestone due dates correspond to deliverable deadlines.

Link to GitHub Project

Schedule

This project is organized into 10 milestones spanning two terms.

- Milestone 1 (Weeks 3-4): Problem Statement, Proof of Concept Plan, Development Plan, Team Charter
- Milestone 2 (Weeks 5-6): Software Requirements Specification, Hazard Analysis (Rev 0), User/Stakeholder/Client Interview Report (Fall Custom Document Rev 0)
- Milestone 3 (Weeks 7-8): Verification and Validation Plan (Rev 0)
- Milestone 4 (Weeks 9-10): Design Document (Rev -1)
- Milestone 5 (Weeks 11-13): Proof of Concept Demo, Interviews
- Milestone 6 (Weeks 14-16): Design Document (Rev 0)
- Milestone 7 (Weeks 17-19): Revision 0 Demo, Interviews
- Milestone 8 (Weeks 20-22): Verification and Validation Report (Rev 0), Usability Testing Report (Winter Custom Document Rev 0)
- Milestone 9 (Weeks 23-24): Final Demonstration (Rev 1)

- Milestone 10 (Weeks 25-26): EXPO Demonstration, Final Documentation (Rev 1)

9 Proof of Concept Demonstration Plan

The main purpose of the solution is to give supervisors a central place to publish projects in a standard format and allow students to find and request available projects with a clear understanding of the project's status. This plan focuses on the solution's greatest implementation risk and demonstrates it in a small, targeted demo rather than a full system.

9.1 Main Risks for the Project

Main Risk(s)	**How it threatens success**	**What will be demonstrated in the PoC**
Project availability and team assignment state may become inconsistent across the project lifecycle and the team-on-project lifecycle defined in the SRS. A team can apply to several projects and a project can accept several teams (up to its capacity), and a single action in one lifecycle has effects on other records. A supervisor accepting a team reserves a slot, which can make the project full and close its other pending applications. A team agreeing to one project withdraws its other applications and releases any slots reserved for it elsewhere. A decline, withdrawal, or rejection releases a slot only if one was reserved.	The platform exists to show accurate availability and status. If these linked effects are not applied together, a project could appear full when a slot is free (or open when it is not), a team could hold slots on several projects, or a supervisor and a team could see different statuses. This would recreate the confusion the platform is meant to remove.	Using seeded data, a scripted series of actions covers the full application, reservation, agreement, and release paths across several teams and projects. After every action, the data is checked against a set of consistency rules, and the student and supervisor views show a synchronized status. Invalid actions are rejected without changing any state.

Table 2: Main project risks and proof of concept demonstrations.

Scope of the PoC: The PoC uses seeded students, teams, supervisors, and projects, and has no authentication or role-based access control. Anyone using the demo can perform any action (e.g., by choosing who they are acting as), but the system still enforces the state rules described below. Team formation, project proposal and listing, notifications, discussion threads, and post-approval changes are excluded. Team formation is similar in logic to project selection, and the rest do not contribute to the risk above. The instructor's review of a team's project is reduced to a simple approve or reject action, and reapplying after a rejection is not covered.

9.2 Proof of Concept Plan

To address the high-priority risk above, the following PoC plan will be demonstrated, using projects of different capacities (e.g. two projects accepting one team and one accepting two) and several teams:

1. Teams apply to several open projects. An application does not use a slot.
2. A supervisor accepts a team's application. A slot is reserved for that team and the number of reserved slots on the project increases. If the project's capacity is reached, the project becomes full and its other pending applications are closed. A supervisor cannot accept a team if no capacity remains.
3. A second supervisor accepts the same team on another project, so the team holds a reserved slot on both projects.
4. The team accepts one of the projects, so an agreement is reached. The team's other applications are withdrawn and the slot reserved on its other project is released, returning that project to open if previously full.
5. Teams that either decline after a supervisor's acceptance, withdraw after an agreement, or are rejected by the instructor each release their slot, allowing a full project to become open again.
6. The instructor approves an agreed team's project (or rejects it, as in the previous step).
7. After each action, the student and supervisor views display a synchronized status (e.g. applied, accepted, agreed, declined, withdrawn, closed), and the consistency rules in Section 9.3 are checked automatically.
8. Views may be briefly out of date. Invalid actions taken from a stale view (e.g. accepting a team whose application was withdrawn, applying to a project that has become full, or submitting the same action twice) are rejected with an error message and leave the state unchanged.

Concurrency. Each action is carried out by the server as a single transaction that checks the state rules and applies all resulting changes together, so a supervisor's capacity check and slot reservation cannot be separated. Correctness is

enforced on the server rather than by refreshing views, so automatic refreshing is a usability improvement for the final system and is not part of the PoC. Stress testing with simultaneous requests is left to the final system's verification and validation.

9.3 Success Criteria

After every action in the PoC plan, the following hold:

- The number of reserved slots on a project equals the number of teams that have been accepted by its supervisor and have not declined, withdrawn, or been rejected.
- The number of reserved slots never exceeds the project's capacity.
- A project is open exactly when its reserved slots are fewer than its capacity, and full otherwise.
- A team that has reached an agreement has exactly one project and holds no reserved slots on any other project.
- A full project has no pending applications, and a team with an agreement has no pending applications.
- Student and supervisor views show the same status once refreshed, and every invalid action from a stale view is rejected without changing state.

9.4 Mitigation Strategy

In the event the PoC fails to maintain consistency in project availability, group assignments, or capacity tracking, the data model or overall assignment workflow will be simplified. Complexity can be reduced by deferring features such as multi-team projects. Constraints can also be relaxed by reducing the amount of automated checks and state updates. An example would be allowing supervisors to manually mark a project as taken.

10 Expected Technology

- **Version Control and Project Management:** Git, GitHub and GitHub projects
- **Frontend:** React – chosen for its large ecosystem, team familiarity, and strong component-based structure suited to an interactive web application. **Trade-off:** steeper initial setup compared to simpler frameworks, but this is offset by better long-term maintainability.
- **Backend:** Python with FastAPI – chosen for fast development speed, built-in data validation, automatic API documentation, and strong support for any data-processing or ML-adjacent work the project may require.

Trade-off: lower raw performance than a compiled-language backend, which is an acceptable trade-off given the project's expected scale.

- **Backend unit testing:** pytest with pytest-cov – chosen over the built-in `unittest` module for its more concise syntax while running tests, fixture system, and better plugin ecosystem for coverage reports generated by pytest-cov
- **Frontend unit testing:** Jest – chosen for its tight integration with React, minimal configuration overhead, and coverage reporting capabilities for JavaScript or TypeScript code
- **E2E Testing:** Playwright – industry standard for end to end testing of web applications
- **Frontend linter:** ESLint – industry standard for TypeScript/JavaScript, configurable to enforce team-agreed style rules.
- **Backend linter:** Ruff – industry standard linter, easiest to setup, fastest and most modern
- **Unit test coverage tool:** SonarQube – industry standard for reporting and visualizing code coverage based on imported coverage reports generated by pytest-cov and Jest
- **Database:** PostgreSQL - relational database, chosen for its strong support for relational data and constraints relevant to our assignment/status-tracking logic.
- **Application Packaging:** Docker - production standard to package application components and dependencies into one executable container **Trade-off:** Extra set up is required but this will ensure our code will run anywhere we choose to deploy in the future
- **Continuous Integration:** GitHub Actions. CI will run on every push and pull request, checking: linting (ESLint/Ruff), unit test pass/fail (Jest/pytest), and SonarQube coverage reports. Jest will generate an LCOV coverage report, while pytest-cov will generate a Cobertura XML report which will both be imported by SonarQube for analysis.
- *Note: the technologies listed here reflect the team's current expectations and are not binding constraints. They may evolve as the project's design becomes more concrete.*

11 Use of AI and LLMs

- **Purpose:** AI will be used to accelerate development and ensure deliverables align with the rubric and checklists, while the team retains ownership of all thinking, design, and logic decisions.

- **Tools:** Claude and ChatGPT for documentation and code implementation assistance, GitHub Copilot or Claude Code for in-editor code assistance.
- **Documentation use of AI:**
  - Brainstorming and idea generation
  - Proofreading, grammar, and spelling checks.
  - Assistance with LaTeX syntax and compilation issues.
  - Checking documents against the rubric and checklist for completeness.

  Documentation content itself will be written by the team, not generated wholesale by AI.
- **Code use of AI:**
  - A walking skeleton of the codebase must be written manually by the team.
  - Function logic should first be written out as pseudocode by a team member; AI may then be used to help implement that pseudocode, rather than generating logic from scratch.
  - AI is not used to make architecture or design decisions – these remain human decisions.
  - All AI-generated code must be reviewed line by line before being committed. Data structure choices, naming conventions, and other design decisions are made by the team, not the AI.
  - AI may assist in generating unit tests and identifying edge cases, but all generated tests must be reviewed and understood by the team, not accepted blindly.
  - Team members must manually verify that AI-generated tests are actually testing the intended behaviour (e.g. by intentionally breaking the implementation and confirming the test fails).
- *Note: this section covers AI's role in building the project. AI use in drafting this document itself is disclosed separately in the Generative AI Use Disclosure appendix.*

12 Coding Standard

The team will follow the coding standards and conventions below to ensure consistency, readability, maintainability, and code quality across the project:

- **Python / FastAPI:** Python code should adhere to **PEP 8**. Ruff is the Python Linter used to enforce these standards and identify common code-quality issues.
- **Typescript / React:** Frontend code will follow consistent Typescript and React conventions for naming, formatting, component structure, and static typing typically enforced by ESLint.
- **Software Design/Architectural Principles: SOLID** and **GRASP** principles should be used to ensure proper design principles are followed such as single responsibility, separation of concerns, high cohesion, low coupling, modularity, abstraction, and extensibility. Following these will allow the project to have a maintainable system design.
- **Testing Standards:** Test design should be guided by ISO/IEC/IEEE 29119-4 to ensure consistent software testing practices based on requirements. The standards cover software testing techniques used to evaluate boundary conditions, edge cases, and expected failure conditions.
- **Statement Coverage:** Automated tests should target approximately 80% statement coverage for application code. Coverage will be measured automatically during CI using Jest and pytest-cov while results are reported through SonarQube. Falling below the 80% coverage target will not automatically fail the CI or block a merge, but will be flagged for review so the team can identify areas where more testing is needed. Only test failures will cause the CI to fail. Statement coverage along with test design standards defined above should determine overall desired test quality.
- **Readability and Naming:** Code should use descriptive and consistent names and favour clear, simple implementations over unnecessary complexity. Comments should give high-level implementation explanations.
- **Error Handling:** Failure conditions should be handled explicitly, with clear and consistent error messages.

13 Appendix — Generative AI Use Disclosure

13.1 AI Tools Used

- ChatGPT — GPT-5.6 Sol
- Claude Sonnet 5

13.2 How and Where Used

- Section 9 - Proof of Concept Demonstration: Brainstorming potential risks with the project, determining if my PoC was done correctly or if it was comparable to an MVP, reviewing and refining PoC section purely to identify errors and inconsistencies.
- Section 12 - Coding Standard: Learning what formal software standards exist for various technologies, querying to learn what would qualify as a coding standard and what would not, reviewing the section for minor errors.
- Used it to check both documents against the checklist and rubric and asked it to flag anything that was missing and any grammar issues
- Goals - used to help check if goals were too abstract or too low level.

13.3 Verification of AI-Generated Content

- Found resources that discuss similar ideas particularly Coding standards (things like GRASP/SOLID)
- Every AI Generated feedback was thoroughly read, revised manually, and was never copy-pasted directly into the document.

Appendix — Reflection

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

Waqar Ul-Hassan:

1. Why is it important to create a development plan prior to starting the project?

   For a project of this size, especially because we are developing it from scratch it is critical that we create a plan to ensure we have a scope of the project. We can set realistic expectations and everyone in the group is on the same page to ensure we don't have any miscommunication. Additionally, this way we are also able to identify risks earlier on by evaluating the project and thinking about it from a more management perspective.

2. In your opinion, what are the advantages and disadvantages of using CI/CD?

   I feel that the biggest advantage of using CI/CD is to have an automated testing suite which will build when you push code and catch bugs, code formatting, etc. This ensure that we are keeping a certain standard of work in our active repository and overall are more confident when pushing to the main branch. The biggest disadvantage I think is definitely the maintenance of the CI/CD pipeline, making sure its up to date. On top of this if the pipeline is not made very will, it can cause some very frustrating problems and become a huge waste of time, especially when trying to scale a project.

3. What disagreements did your group have in this deliverable, if any, and how did you resolve them?

   Initially we had some disagreements on things like, which backend language we should with our react frontend. One of my group members wanted to use nextJS while others wanted to use python FastAPI. Another, disagreement we had was when deciding on the metric to use to check for contribution amount to the project. Some wanted to use lines

of code, commits, and others wanted to use number of issues. For all these initial disagreements, we just held discussions during our meetings. In these discussions, we weighed the pros and cons of using each of the different methodologies. After weighing them together, we then decided for example to use python FastAPI as we feel we have more experience with it and also its much more flexible as well as use the number of issues as our comparison metric.

Ayush Patel:

1. Why is it important to create a development plan prior to starting the project?

   This is an 8-month project with 4 team members, and everyone has different experience levels, expectations, and work styles when it comes to project management. It's best to get everyone on the same page early about things like how we're going to use Git and GitHub (branching, issues, milestones, etc.), and what our expectations are around meetings and attendance.

   Documenting all of this early and getting everyone to agree to it means we have something concrete to point back to later if something goes wrong or if someone isn't pulling their weight.

   I also really like the external goals section of the development plan, since everyone has different things they personally want to get out of the capstone, and it's good to be transparent about that early rather than finding out halfway through.

2. In your opinion, what are the advantages and disadvantages of using CI/CD?

   Advantages: regression bugs get caught early and often, on every push to a PR and every merge to master, instead of piling up and surfacing all at once near the deadline.

   Disadvantages: CI can give a false sense of confidence. A green build doesn't necessarily mean everything actually works, it just means your tests passed, and if your tests aren't well designed, a green checkmark can hide real bugs. Setting up CI by itself doesn't automatically catch regressions, you still have to actually design your unit, integration, and e2e tests properly so they're checking the functionality that actually matters. This is why it is very important to not only run your CI frequently, but also to maintain and update the test suite and other checks that the CI is running.

3. What disagreements did your group have in this deliverable, if any, and how did you resolve them?

   We had a disagreement about what backend language/framework to use. Some team members wanted Next.js, but most of us wanted Python be-

cause it's easy to use and has great library support if we need it down the line. We explained our rationale to the rest of the group and everyone agreed to go with Python.

We also disagreed on what to use for productivity metrics, things like commit count, number of issues closed, or cumulative issue size. We were all a bit unsure since no single metric really captures productivity well. What we landed on is that we'll create issues for each deliverable, try to split the actual work evenly (not just the number of issues), and use hitting the deadline as our real signal for staying on track. We got there by weighing the pros and cons of the other options and picking whatever made the most sense for how our team actually works.

Ahmed Rashrash:

1. Why is it important to create a development plan prior to starting the project?

   Drawing on previous experience, when projects get large enough and has multiple collaborators, many points of contention and slow-downs occur as a consequence of the development process itself. Differences in code styles, tool usage, quality expectations, workflows, and documentation can result in expectations not being met, and issues being raised as a result which need to be resolved using reviews, discussions, and meetings, where they weren't needed if expectations were setup properly. Creating a development plan from the start can reduce future strain on a project's progress and allow the time saved to be spent on improvements and expanding capabilities.

2. In your opinion, what are the advantages and disadvantages of using CI/CD?

   CI/CD can bring a standard of quality that manual review simply cannot do reliably. Automated tests, static analysis tools, and artifact generation can be completed in the background while important tasks such as design and implementation receive full focus and attention. CI/CD can raise a notification only when something requires manual intervention (failed test, build, etc.), reducing noise in the development process. While CI/CD can bring immense value, CI/CD itself can grow to be a system of its own requiring setup, debugging, and cloud costs, all of which is additional overhead that must be justified.

3. What disagreements did your group have in this deliverable, if any, and how did you resolve them?

   The main disagreements in this deliverable were against some of the expected technologies and contribution quality. In terms of expected technologies, we weighed the value of familiarity with certain technologies against the potential of a better developer experience with a more unfamiliar technology. We resolved this disgreement through a meeting by

laying out pros and cons, what considerations matter when selecting technologies, and were able to come to a consensus without leaving it up to a vote. Similarly with contribution quality and metrics, we were able to come to a consensus.

Andrew Habib:

1. Why is it important to create a development plan prior to starting the project?

   A development plan is important because it gives the group a pre-made roadmap or contract that will guide how the project will be scheduled, built, tested, reviewed, and demonstrated. It ensures that we are aligned on the overall picture of how we aim to organize critical tasks and what methods we will use to execute them. Things like technologies, team roles, meeting plans, version control, coding standards, and PoC are critical items to ensure that we all follow a unified process and build the application using the best practices as a team without any confusions or misalignments.

2. In your opinion, what are the advantages and disadvantages of using CI/CD?

   The advantage of CI/CD is that it serves as an automated method to build, test, and deploy code or documentation. It helps retain consistency and repeatability when working through our milestones as the same pipelines ensure that our work is valid and passes all requirements so our work meets the highest standards. At the same time, CI/CD pipelines are complex to setup and require some time to ensure that they work as intended. On top of this, there is maintenance overhead that comes with the pipelines and minor failures can sometimes halt project progress.

3. What disagreements did your group have in this deliverable, if any, and how did you resolve them?

   The disagreements that we had for this deliverable were mainly related to the project technology stack. This is due to our differing expertise in full stack development. An example of this was that some people were familiar with Python FastAPI for the backend while others were more comfortable with using either Node/Express or Next.js. Resolving this required open-mindedness to learn and discover new technologies since many technologies could work for our use-case. We evaluated the varying technologies and ultimately decided that Python was the most versatile language and could give us flexibility to work with many different libraries we may need down the line.

Appendix — Team Charter

External Goals

- Produce something we are proud of and can discuss meaningfully in interviews.
- Given the nature of the project, aim for it to be genuinely useful to future capstone students and course staff after we graduate.
- Improve our project management, design, and technical skills by applying them to a real-world problem.
- Get an A+

Attendance

Expectations

- 80% attendance is required for team, TA, and supervisor meetings.
- If a member must miss a meeting, they must notify the team in advance and provide a written update covering their progress and any action items relevant to them.
- Members attending a meeting are expected to arrive on time (at most 10 minutes late) and stay for its full duration.
- **Trigger:** Missing 2 consecutive meetings, *or* being late thrice in a row, triggers a consequence.
- **Consequence:** the member brings snacks/coffee to the next team meeting for a first occurrence; a second occurrence within the same term triggers a mandatory check-in meeting with the TA to discuss the pattern.
- These standards apply to non-emergency cases only (see below for emergencies).

Acceptable Excuse

- **Acceptable:** midterm preparation, conflicting lectures/tutorials, work-related obligations, medical appointments.
- **Not acceptable:** forgetting the meeting, non-urgent personal plans, or repeated last-minute excuses of the same kind. Repeated use of the same excuse will be treated as a pattern and addressed under the "Stay on Track" consequences below.
- Advance notice is expected wherever the excuse is foreseeable (e.g. a scheduled appointment).

In Case of Emergency

- In a genuine emergency, the attendance rules above do not apply.
- The member should notify the team as soon as reasonably possible.
- If the emergency will materially impact a deliverable, there are two options:
  1. The other members will redistribute the member's tasks among themselves, and then that team member can pick up more work in future deliverables.
  2. If the rest of the team cannot handle the extra workload, the team will raise it with the TA and/or course instructor proactively, rather than waiting until the deadline is missed.

Accountability and Teamwork

Quality

- Deliverables must follow the AI usage guidelines in this document – no unreviewed "AI slop."
- Every deliverable is checked against both the Avenue rubric and the repository checklist before submission, by the assigned documentation QA role for that deliverable (see Team Member Roles) in addition to the author.
- Members are expected to arrive at meetings ready to give a meaningful update or raise questions.
- Members should stay reasonably current on lecture content and upcoming deliverable requirements.

Attitude

- Team members are expected to be positive, respectful, and willing to work hard.
- All members' ideas are given fair consideration; disagreement is handled respectfully, not dismissively.
- Members are expected to communicate openly rather than let frustration build silently.

Stay on Track

- Progress is tracked via the GitHub Project board – issue status, assignee, and milestone fields are kept current so contribution is visible to the whole team, not just self-reported.

- **Regular Contributions:** Each member should have visible, regular activity (commits, PRs, or issue updates) across each week of an active deliverable period, not concentrated entirely in the final 1–2 days.
- **Target metrics:** Issues will be assigned at the beginning of each deliverable or during each weekly meeting. We will try to assign each person a relatively equal amount of work and finishing those issues within the provided deadline will be the main target metric.
- **Consequence for falling behind and repeated excuses:** First instance – a direct, informal conversation with the member to understand blockers and rebalance tasks; a repeated pattern across multiple deliverables will result in a joint conversation involving the TA.

Team Building

We will play ping pong together.

Decision Making

- Decisions are made by consensus where possible.
- Where consensus cannot be reached, the team will vote; the majority decision stands.
- Where a disagreement cannot be resolved by voting, the team will default to whichever option best aligns with the rubric and industry best practice; if still unresolved, the TA or supervisor will be brought in to mediate.